arXiv:2607.17913v1 [cs.DC] 20 Jul 2026

AutoEncoder-Compressed Parallel Split Learning for Pre-trained Model Fine-Tuning

Bas Meuwissen (✉), Vasileios Tsouvalas, and Nirvana Meratnia

Eindhoven University of Technology, Eindhoven, The Netherlands
b.meuwissen@student.tue.nl, {v.tsouvalas,n.meratnia}@tue.nl

Abstract. Distributed Fine-Tuning (DFT) of large-scale Foundation Models (FMs) on resource-constrained edge devices is limited by local compute constraints and communication overhead. Parallel Split Learning (PSL) reduces client-side computation by keeping few model layers on each client and offloading the remaining computation to the server; however, clients must exchange intermediate activations and gradients with the server at every training step. Existing SL communication-compression methods mainly rely on task-agnostic heuristics, such as sparsification and quantization. While learnable SL compressors can better adapt to intermediate representations, they require co-training with the target model. Therefore, directly inserting them into off-the-shelf FMs introduces feature-distribution misalignment and degrades DFT performance. To address this, we propose `AE-PSL`[1], a communication-efficient PSL framework that compresses intermediate activations and gradients using a lightweight AutoEncoder (AE) placed at the split layer. To ensure compatibility of AE compression with pre-trained FMs, `AE-PSL` introduces a novel two-stage alignment mechanism, which adapts the AE to the pre-trained model's feature manifold and client-specific feature distributions before DFT. We evaluate `AE-PSL` across four vision datasets and various communication reduction settings, showing that it preserves downstream accuracy at 10.2× reduction; at the same communication budget, the strongest evaluated heuristic baseline trails `AE-PSL` by 5.2 percent point. This demonstrates that `AE-PSL` offers a stronger accuracy–communication trade-off than heuristic SL compression.

Keywords: parallel split learning · autoencoder compression · distributed fine-tuning · foundation models

1 Introduction

Large-scale transformer-based Foundation Models (FMs) have established state-of-the-art performance across different modalities and complex tasks. These models are trained on a massive amount of data gathered on a central server. Meanwhile, the proliferation of Internet of Things (IoT) devices has led to a vast amount of data residing at the edge. Such data can be valuable for fine-tuning

[1] Public code repository: `https://github.com/Nousphera/AE_PSL`

FMs, yet centralizing it is often infeasible due to privacy constraints, while local fine-tuning is limited by the memory and compute demands of large models.

To address these constraints, distributed learning schemes have been explored. Federated Learning (FL) [24] enables collaborative FM fine-tuning while keeping client data localized; yet it requires each client to train the full model, imposing substantial storage and computational burdens on edge devices. Split Learning (SL) [13] alleviates this by splitting the model at a cut layer into client-side and server-side sub-models, allowing clients to execute only the *lightweight* client-side component. Recent Parallel SL (PSL) variants, such as Scalable Aggregated SL (SASL) [21], further improve SL's scalability through parallel client training and reduced server-side computation. However, unlike FL's round-based model-update exchange, SL requires clients to transmit intermediate activations and receive their corresponding gradients at every training iteration. This shifts the main cost from computation to communication, especially in transformer-based models whose intermediate tensors scale with sequence length [36,30].

Existing SL communication-compression methods mostly rely on heuristics, such as quantization [8,26] and pruning [5,38,1], which are simple but task-agnostic and can discard information critical to pre-trained FM representations. Learnable compression methods, including AutoEncoder (AE)-based modules, have shown promise in intermediate representation compression [10,23,28,2,12,29,7,25]. However, prior AE-based methods target split-computing inference or jointly train the model with the AE module, making them ill-suited for fine-tuning off-the-shelf pre-trained FMs, where a randomly-initialized AE can distort intermediate representations and degrade performance.

To bridge this gap, we introduce `AE-PSL`, a scalable PSL framework for communication-efficient fine-tuning of off-the-shelf pre-trained FMs. `AE-PSL` places a lightweight AE module at the split layer, with its encoder on the client and the decoder on the server, reducing the bidirectional communication overhead of activations and gradients. To stabilize fine-tuning and to avoid representation distortion of FMs, we introduce an initial two-stage alignment process that adapts the AE module to general and client-specific feature distributions, without re-training the original model from scratch or centralizing private data. Our contributions are as follows:

- We introduce `AE-PSL`, an AE-compressed PSL framework for fine-tuning pre-trained vision FMs, reducing both activation and gradient communication through client-side encoders and a shared server-side decoder.
- We design a 2-stage alignment protocol, comprising General Alignment (GA) and Client-Specific Alignment (CSA), that adapts the AE module to the pre-trained model and client-specific feature distributions, enabling stable integration with off-the-shelf FMs without centralizing private data.
- We evaluate `AE-PSL` across four vision datasets, showing that feature-aligned learnable compression provides a more robust accuracy–communication trade-off than heuristic SL methods, yielding $\sim 10\times$ communication reduction with no accuracy degradation and negligible client-side computational overhead.

2 Related Work

Distributed Training and PEFT. Efficient FM adaptation on edge devices requires reducing client-side computation while avoiding centralized access to private data. FL preserves data locality, but requires clients to train the full model, imposing prohibitive memory and compute demands on resource-constrained devices [24]. SL addresses this by splitting the model between clients and a server [13], while scalable variants such as SASL further enable parallel client training and reduce server-side backpropagation to $\mathcal{O}(1)$ through global loss aggregation [21]. Orthogonally, PEFT methods such as LoRA [15] have been integrated into Split FL (SFL) to reduce trainable parameters and weight-updates overhead during fine-tuning [19,33,6,18,35,20,22,34]. However, PEFT primarily targets model-parameter efficiency and does not reduce the per-iteration exchange of intermediate activations and gradients, which remains a major communication cost in transformer-based SL [36,30,11]. `AE-PSL` addresses this complementary cost by compressing intermediate representations within scalable PSL, while retaining split execution and PEFT-based fine-tuning.

Communication Compression in SL. In SL, communication reduction for intermediate representations typically relies on heuristic or learned mechanisms. *Heuristic compression* approaches rely on predefined, non-parametric operations such as quantization [8] or sparsification (e.g. top-k) [5]. To enhance convergence, Rand-Top-K [38] extends standard top-k selection by randomly transmitting a fraction of tokens outside the top-k threshold. Attention-based Double Compression (ADC) [1] performs batch-wise compression by clustering sample activations with K-means using class-token attention scores, averaging activations within each cluster, and then retaining only the top-k spatial tokens most aligned with each cluster centroid. C3-SL [14] relies on algebraic compression, using circular convolution and superposition to encode batches of high-dimensional intermediate features into compact vectors that the cloud server reconstructs via circular correlation with fixed quasi-orthogonal random keys.

Although heuristic compression methods are simple and effective at moderate reduction levels, their fixed, task-agnostic rules cannot adapt to the structure of intermediate representations and may discard features that are important for downstream prediction. This motivates *learnable compression*, which trains the encoding scheme to preserve task-relevant information under communication constraints. BottleNet [10] uses a convolutional AE-based architecture at the cut layer to reduce intermediate image features, while BottleNet++ [28] extends this design with joint source–channel coding to improve robustness under unreliable communication. BottleFit [23] replaces selected layers of a pre-trained model with a bottleneck module and uses a two-stage training procedure to first learn compressed representations and then adapt downstream layers to them. These methods focus on *split computing*, where only forward inference is partitioned, rather than SL, where compressed intermediate representations must support end-to-end training across the cut layer. Pioneering works implement learnable compression in SL, however these require joint training from scratch and are designed for convolutional networks [7,12,2,25,29]. We address these limitations with

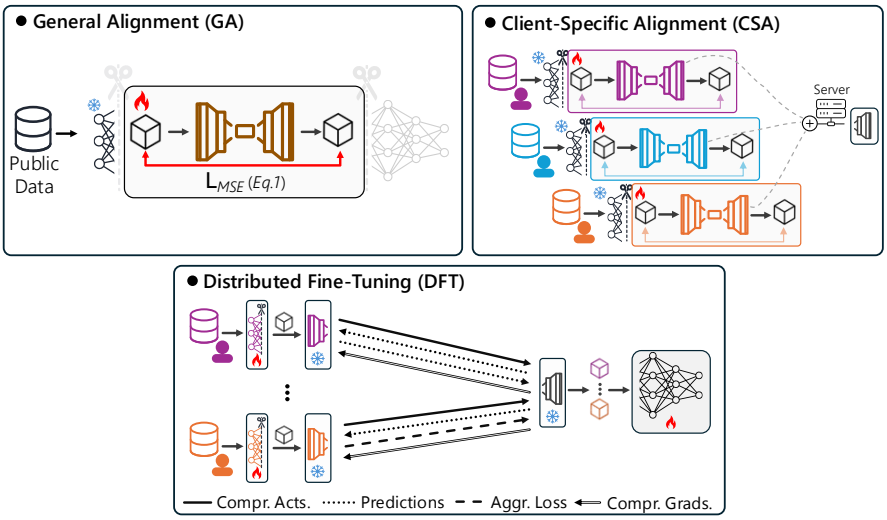


Fig. 1: Overview of `AE-PSL`, consisting of three sequential stages: (1) *GA* trains the AE on public data to match the pre-trained model's feature manifold. (2) *CSA* locally adapts the AE to client-specific data, aggregating only the decoders at the server. (3) *DFT* executes distributed training with frozen AEs, minimizing communication overhead by transmitting compressed activations and gradients.

an AE-based mechanism tailored to pre-trained feature distributions, enabling communication-efficient fine-tuning of FMs within scalable, parallel, label-private SL regimes.

3 Background

While SFL parallelizes client-side training, it imposes an $\mathcal{O}(n)$ computational and memory burden on the server, which must maintain distinct per-client sub-models and execute separate backpropagation passes. Scalable Aggregated Split Learning (SASL) [21] resolves this bottleneck by reducing server-side backpropagation complexity to $\mathcal{O}(1)$ via global loss aggregation. Under the SASL framework, clients forward their local intermediate activations to a single, shared server model. The server processes these activations during the forward pass and returns predictions to the clients. Crucially, clients compute their local losses using private labels and transmit only the scalar loss values back to the server. The server aggregates these individual losses into a single global loss function to execute a single backpropagation pass. The resulting cut-layer gradients are then distributed back to the clients to complete local backpropagation. This workflow eliminates per-client server-side sub-models while maintaining label privacy.

4 `AE-PSL`: Communication-efficient PSL via AutoEncoder

To mitigate SL communication bottlenecks, we propose an AE-based module that compresses intermediate activations and gradients in PSL.

Notation. We denote scalars by x, vectors by $\mathbf{x}$, matrices by $\mathbf{X}$, sets by $\mathcal{X}$, and neural networks by $f_{\boldsymbol{\theta}}$, where $\boldsymbol{\theta}$ denotes their parameters. Let $\mathcal{N}$ be a set of N clients, each with local dataset $\mathcal{D}_n$ from which mini-batches $\mathcal{B}_n \subset \mathcal{D}_n$ are sampled, and let $\mathcal{B} = \bigcup_{n\in\mathcal{N}} \mathcal{B}_n$ denote the global batch. A pre-trained transformer $f_{\boldsymbol{\theta}}$ is split at layer s into client-side and server-side components $f_{\boldsymbol{\theta}_c}$ and $f_{\boldsymbol{\theta}_s}$. For client n, the client-side model outputs smashed data $(\mathbf{h}_{\text{cls},n}, \mathbf{H}_n)$, consisting of the `[CLS]` activation and patch-token matrix, respectively.

Problem Formulation. To reduce communication, AE-PSL inserts a learnable AE at the split point: the client-side encoder $E_{\boldsymbol{\phi}}$ compresses $\mathbf{H}_n$ from hidden dimension d to latent dimension $d_z \ll d$, producing $\mathbf{Z}_n = E_{\boldsymbol{\phi}}(\mathbf{H}_n)$, while the server-side decoder $D_{\boldsymbol{\psi}}$ reconstructs $\widehat{\mathbf{H}}_n = D_{\boldsymbol{\psi}}(\mathbf{Z}_n)$. Our objective is to fine-tune $(f_{\boldsymbol{\theta}_c}, f_{\boldsymbol{\theta}_s})$ while preserving task-relevant information. To minimize communication overhead during this process, we transmit the compressed representations $\mathbf{Z}_n$ and their gradients instead of the raw intermediate activations $\mathbf{H}_n$.

4.1 Two-Stage AutoEncoder Alignment

A key challenge in using AEs with pre-trained models is avoiding catastrophic degradation caused by feature-distribution misalignment [10]. We address this through an initial two-stage alignment procedure, aligning the AE to client feature distributions while requiring no server-side access to private downstream data.

4.1.1 General Alignment (GA). We first insert the AE at the cut layer s of the pre-trained model $f_{\boldsymbol{\theta}}$ and train it on the server-side using publicly available data (e.g., ImageNet [27]), $\mathcal{D}_{\text{p}}$, to reconstruct split-layer activations, while keeping $f_{\boldsymbol{\theta}_c}$ frozen. Specifically, we minimize the reconstruction MSE:

$$(\boldsymbol{\phi}^{\text{GA}}, \boldsymbol{\psi}^{\text{GA}}) = \arg\min_{\boldsymbol{\phi},\boldsymbol{\psi}} \tfrac{1}{|\mathcal{D}_{\text{p}}|} \textstyle\sum_{\mathbf{x}\in\mathcal{D}_{\text{p}}} \|\mathbf{H}(\mathbf{x}) - D_{\boldsymbol{\psi}}(E_{\boldsymbol{\phi}}(\mathbf{H}(\mathbf{x})))\|_2^2 \ , \tag{1}$$

where $\mathbf{H}(\mathbf{x})$ denotes the patch-token activations extracted at split layer s from sample $\mathbf{x}$, with $f_{\boldsymbol{\theta}_c}$ frozen. Thus, GA is a one-time alignment step for a given pre-trained model that adapts the AE to the model's feature manifold.

4.1.2 Client-Specific Alignment (CSA). While GA aligns the AE with the feature manifold of the pre-trained model, local client data can induce task-specific and non-independent and identically (non-IID) distributed feature shifts [37]. To this end, each client n receives the GA-initialized pair $(\boldsymbol{\phi}^{\text{GA}}, \boldsymbol{\psi}^{\text{GA}})$ and locally warms up the AE on $\mathcal{D}_n$ using the same MSE reconstruction objective as in Eq. 1, while keeping $f_{\boldsymbol{\theta}_c}$ frozen. Crucially, CSA is fully local and lightweight, requiring only the frozen client-side partition and AE module, with no forward passes over the $f_{\boldsymbol{\theta}_s}$ sub-model.

To preserve scalability, we avoid per-client server decoders. Instead, after CSA, each client keeps its adapted encoder $\boldsymbol{\phi}_n$ locally and sends only $\boldsymbol{\psi}_n$ to the server, which aggregates them by weight averaging (i.e., $\boldsymbol{\psi}^{\text{global}} = \frac{1}{N}\sum_{n\in\mathcal{N}} \boldsymbol{\psi}_n$). During the subsequent distributed fine-tuning phase, the server utilizes this aggregated decoder, $D_{\boldsymbol{\psi}^{\text{global}}}$, while each client employs its locally adapted encoder. This

setup aligns the compression module with heterogeneous client distributions without sacrificing the scalability of a unified server-side architecture.

4.2 AutoEncoder-Compressed Distributed Fine-Tuning (DFT)

During DFT, client n retains its encoder $E_{\boldsymbol{\phi}_n}$ after CSA, while the server uses the aggregated decoder $D_{\boldsymbol{\psi}^{\text{global}}}$. With both AE modules *frozen*, the fine-tuning procedure is as follows:

1. *Client FP:* Client n computes smashed data $(\mathbf{h}_{\text{cls},n}, \mathbf{H}_n)$, compresses $\mathbf{H}_n$ as $\mathbf{Z}_n = E_{\boldsymbol{\phi}_n}(\mathbf{H}_n)$, and sends $(\mathbf{h}_{\text{cls},n}, \mathbf{Z}_n)$ to the server.
2. *Server FP:* The server reconstructs $\widehat{\mathbf{H}}_n = D_{\boldsymbol{\psi}^{\text{global}}}(\mathbf{Z}_n)$ and forwards $(\mathbf{h}_{\text{cls},n}, \widehat{\mathbf{H}}_n)$ through $f_{\boldsymbol{\theta}_s}$ to obtain predictions $\hat{\mathbf{y}}_n$.
3. *Loss Aggregation:* Each client computes $\mathcal{L}_n$ using its local labels, and the server aggregates the received losses as $\mathcal{L}_g = \sum_{n \in \mathcal{N}} \mathcal{L}_n$.
4. *Server BP:* The server performs a single backpropagation pass, updates $f_{\boldsymbol{\theta}_s}$, and returns $\nabla_{\mathbf{Z}_n} \mathcal{L}_g$ and $\nabla_{\mathbf{h}_{\text{cls},n}} \mathcal{L}_g$ to client n.
5. *Client BP:* Client n backpropagates via the frozen encoder and updates $f_{\boldsymbol{\theta}_c}$.

Since both $\mathbf{Z}_n$ and $\nabla_{\mathbf{Z}_n} \mathcal{L}_g$ operate in the reduced dimension $d_z \ll d$, AE-PSL reduces communication bi-directionally during DFT. Note that, while `AE-PSL` leverages SASL for its server-side efficiency, label privacy, and elimination of per-client sub-models (see Sec. 3), learnable compression inherently generalizes to other SL paradigms. The GA phase optimizes the AE for the general activation manifold of a pre-trained model, a structural property completely independent of the distributed training setup. Furthermore, although CSA is designed to aggregate decoders across parallel clients (as in SFL or SASL), it is equally applicable to standard sequential SL. In a sequential regime, the server can simply aggregate the client-adapted decoders at the conclusion of the local warm-up phase, establishing a unified global decoder that services all subsequent sequential fine-tuning rounds. Consequently, `AE-PSL` serves as a versatile communication-reduction solution applicable to any split-based fine-tuning workload.

5 Experiments

5.1 Datasets & Models

We evaluate `AE-PSL` on four public vision classification datasets, i.e., CIFAR-100 [17], Food101 [3], SUN397 [31], and FEMNIST [4]. We partition client data using Dirichlet sampling [32] with $\alpha = 0.5$ for all datasets except FEMNIST, which provides client-specific non-IID distributed splits. Unless stated otherwise, we use an ImageNet-pretrained ViT-B/32 [9,27] partitioned at layer $s = 5$ in our experiments. DFT is performed for 10 epochs using LoRA [15] with rank $r = 16$ and $\alpha = 32$, a global batch size of 125, and Adam with learning rate 10^{-4}. For GA, the AE is trained on ImageNet100 [27] for 30 epochs. Due to space limitations, we provide the full experimental configuration in Appx. A.1, and analyze alternative split layers in Appx. A.6.

AutoEncoder Architecture. To balance reconstruction fidelity against client-side overhead, we evaluate two architecture types: token-wise *MLP-based AEs* and spatial *Convolution-based AEs*. Structurally native to transformers, *MLP-based AEs* compress each token independently along the hidden dimension d, preserving sequence structure without spatial restructuring. Conversely *Convolution-based AEs* reshape the 1D patch-token sequence into a 2D spatial grid prior to encoding to exploit localized spatial redundancies. This approach is inspired by existing SL learnable compression frameworks designed for convolutional networks [10,23,2]. For MLP variants, we implement a symmetric configuration with the encoder denoted as ℓ-layer-MLP-(d-u_1-$\cdots$-d_z), where ℓ is the layer count, u_i are intermediate widths, and d_z is the compressed latent dimension. These layers incorporate Layer Normalization and `GELU` activations, which empirically outperformed `ReLU` (Appx. A.7.1). Unless specified otherwise, we use a 2-layer-MLP-(d-d-d_z) architecture; detailed AE architectures analysis is in Sec. 5.3.3.

5.2 Evaluation Setup

Scenarios. Due to the split structure of PSL, DFT produces multiple client-side sub-models together with a single shared server-side sub-model, enabling different post-training evaluation modes. We consider two paradigms: *global* evaluation, where client-side weights are aggregated into a single global client sub-model, combined with the shared server-side sub-model, and evaluated on a centralized test set with the AE bypassed; and *local* evaluation, where each client is evaluated using its own client-side sub-model and local test data, with the AE active, and final performance is averaged across clients. Thus, *global* evaluation isolates the effect of compression during DFT and tests whether the model remains compatible with the original uncompressed representation space at inference, while *local* evaluation captures an end-to-end distributed setting where the communication bottleneck remains active during inference. Accordingly, we use FEMNIST for *local* evaluation, as its user-partitioned data provides a separate test set for each client, enabling evaluation under client-specific distributions, while CIFAR-100, Food101, and SUN397 are used for *global* evaluation as centralized image-classification benchmarks.

Baselines. We compare `AE-PSL` against related SL heuristic-based communication-compression methods, covering batch-wise compression (ADC [1], C3-SL [14]) and top-k sparsification (Rand-Top-K [38]). For a controlled comparison, we integrate all baselines into the SASL framework, although the original methods were evaluated under traditional SL or SFL settings; for each baseline, the `[CLS]` token is left uncompressed when doing so improves accuracy.

Metrics. We report downstream task accuracy, client-side computational overhead measured in GFLOPs, and communication reduction R, defined relative to uncompressed DFT. For fair comparison, we evaluate each method 3 reduction levels: *Low* ($R \approx 5\times$), *Mid* ($R \approx 10\times$), and *High* ($R \approx 20\times$), spanning from light compression to a high compression ratio where baselines experience significant degradation (see Sec. 5.3). Note that R is upper-bounded by a hard ceiling, as the

Table 1: Performance under ***global*** and ***local*** evaluation using ViT-B/32 [9] across datasets for various N and R. We report average accuracy across 3 seeds, whereas *deltas* denote relative performance to the *no compression* baseline (R=1) for the same settings. Results with standard deviations are reported in Table 4.

	Dataset	Method	$N = 5$			$N = 25$		
			Low ($R \approx 5$)	Mid ($R \approx 10$)	High ($R \approx 20$)	Low ($R \approx 5$)	Mid ($R \approx 10$)	High ($R \approx 20$)
Global Eval.	**CIFAR100**	No Compr. ($R = 1$)		82.7			82.0	
		C3-SL	66.5 (−16.2)	50.9 (−31.8)	37.3 (−45.4)	60.0 (−22.0)	60.3 (−21.7)	59.9 (−22.1)
		Rand-Top-K	79.9 (−2.8)	76.2 (−6.5)	61.5 (−21.2)	78.8 (−3.2)	75.4 (−6.6)	63.3 (−18.7)
		ADC	78.0 (−4.7)	68.4 (−14.3)	53.7 (−29.0)	71.7 (−10.3)	68.0 (−14.0)	63.4 (−18.6)
		AE-PSL (Ours)	**83.0** (+0.3)	**82.7** (+0.0)	**81.4** (−1.3)	**82.7** (+0.7)	**82.7** (+0.7)	**81.1** (−0.9)
	Food101	No Compr. ($R = 1$)		69.1			67.5	
		C3-SL	37.0 (−32.1)	40.3 (−28.8)	32.1 (−37.0)	31.3 (−36.2)	31.3 (−36.2)	31.3 (−36.2)
		Rand-Top-K	66.5 (−2.6)	60.2 (−8.9)	35.7 (−33.4)	65.1 (−2.4)	60.3 (−7.2)	43.1 (−24.4)
		ADC	63.3 (−5.8)	54.6 (−14.5)	44.0 (−25.1)	56.2 (−11.3)	51.8 (−15.7)	47.1 (−20.4)
		AE-PSL (Ours)	**70.3** (+1.2)	**69.6** (+0.5)	**65.3** (−3.8)	**68.7** (+1.2)	**68.1** (+0.6)	**64.3** (−3.2)
	SUN397	No Compr. ($R = 1$)		67.0			64.5	
		C3-SL	48.0 (−19.0)	41.3 (−25.7)	31.6 (−35.4)	41.5 (−23.0)	41.5 (−23.0)	41.5 (−23.0)
		Rand-Top-K	66.3 (−0.7)	62.9 (−4.1)	48.5 (−18.5)	64.6 (+0.1)	61.2 (−3.3)	50.0 (−14.5)
		ADC	62.2 (−4.8)	52.9 (−14.1)	39.0 (−28.0)	55.5 (−9.0)	52.1 (−12.4)	47.5 (−17.0)
		AE-PSL (Ours)	**67.4** (+0.4)	**66.7** (−0.3)	**64.8** (−2.2)	**65.2** (+0.7)	**65.0** (+0.5)	**63.8** (−0.7)
Local Eval.	**FEMNIST**	No Compr. ($R = 1$)		87.9			85.8	
		C3-SL	85.2 (−2.7)	83.5 (−4.4)	81.4 (−6.5)	80.7 (−5.1)	39.8 (−46.0)	6.9 (−78.9)
		Rand-Top-K	87.6 (−0.3)	87.2 (−0.7)	86.4 (−1.5)	84.6 (−1.2)	83.8 (−2.0)	82.2 (−3.6)
		ADC	86.6 (−1.3)	81.1 (−6.8)	71.8 (−16.1)	82.4 (−3.4)	78.1 (−7.7)	73.8 (−12.0)
		AE-PSL (Ours)	**88.1** (+0.2)	**88.0** (+0.1)	**87.9** (+0.0)	**85.9** (+0.1)	**85.5** (−0.3)	**84.6** (−1.2)

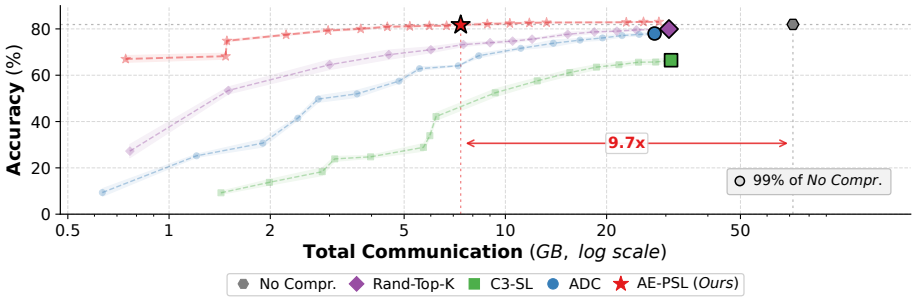
uncompressed `[CLS]` token requires a fixed per-iteration communication budget. All results are averaged over 3 random seeds.

5.3 Results & Discussion

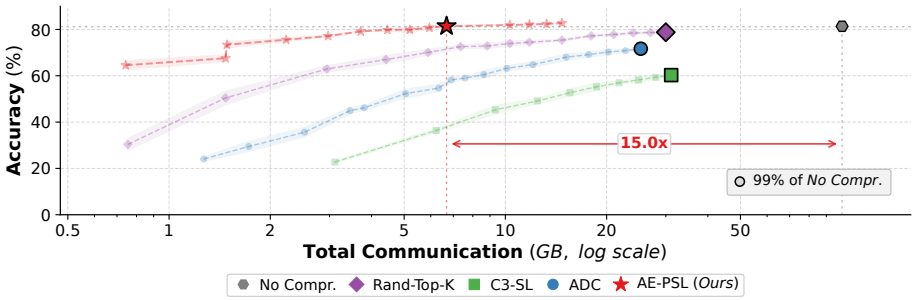
5.3.1 Accuracy–Communication Trade-off

We first examine the central accuracy–communication trade-off: whether each method preserves downstream performance as communication is reduced. We evaluate all methods under *global* and *local* evaluation using ViT-B/32, with $N \in \{5, 25\}$ clients and $R \in \{5, 10, 20\}$. Table 1 shows that `AE-PSL` remains close to the *no compression* baseline across datasets, client counts, and reduction levels. At low and mid reductions ($R \approx 5$ and $R \approx 10$), `AE-PSL` often matches or exceeds the uncompressed baseline, suggesting that the AE bottleneck can regularize intermediate representations by suppressing task-irrelevant noise while preserving semantic features. Even at high reduction ($R \approx 20$), the largest drop is 3.8 percentage points (pp.).

Compared to the baselines, `AE-PSL` achieves the best accuracy–communication trade-off, attaining the highest accuracy across both *global* and *local* evaluation scenarios for all compression levels. Under *global* evaluation, where the AE is bypassed at inference, `AE-PSL` consistently preserves compatibility with the uncompressed intermediate representation space, while heuristic baselines degrade



(a) $N = 5$



(b) $N = 25$

Fig. 2: Accuracy–communication trade-off using ViT-B/32 on CIFAR-100. We report average accuracy across 3 seeds for different communication volumes (GB); markers indicate *closest* (within 99%) of *no compression* DFT.

sharply as R increases, especially on Food101 and SUN397. Under *local* evaluation on FEMNIST, performance of baselines is less affected at $R \approx 5$, yet the gap widens at higher compression levels. Specifically, `AE-PSL` remains within 1.21 pp. of the uncompressed baseline at $R \approx 20$, whereas ADC and C3-SL degrade substantially, with C3-SL collapsing for $N = 25$. Since the global batch is fixed at 125, increasing N from 5 to 25 reduces each clients' mini-batch size; yet, these batch-wise compression methods do not systematically worsen, suggesting that their degradation is driven more by static compression than merely by batch size. Overall, these results highlight that *learnable AE-based compression provides a more robust accuracy–communication trade-off than heuristic compression* across both evaluation modes: it preserves a deployable global model when the AE is bypassed after DFT, while maintaining client-specific accuracy when the communication bottleneck remains active at inference.

Fixed Communication Budget. We further evaluate accuracy as a function of total communication volume across clients. As shown in Fig. 2 for CIFAR-100, `AE-PSL` reaches near-peak accuracy substantially earlier than all baselines for both $N = 5$ and $N = 25$. Specifically, across all datasets and client settings, `AE-PSL` reaches no-compression DFT accuracy with 10.2× less communication on average; at the same communication budget, the strongest evaluated baseline,

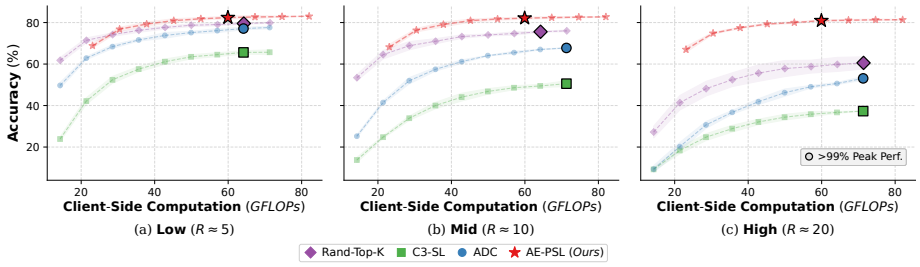


Fig. 3: Accuracy–client-side compute trade-off using ViT-B/32 for $N = 5$ and $R \in \{5, 10, 20\}$ on CIFAR-100. We report average accuracy across 3 seeds for cumulative client-side computation (GFLOPs) across compression levels; markers indicate when a method reaches $> 99\%$ of its peak performance.

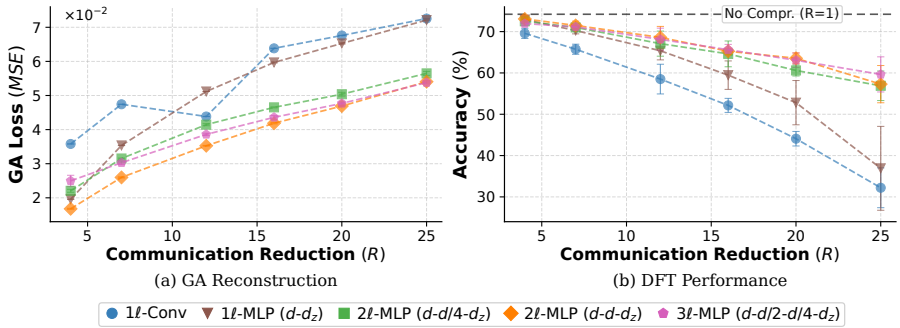


Fig. 4: Effect of AE architecture in `AE-PSL` across communication reduction levels (R). Experiments are conducted with ViT-B/32 [9] on Food101 with $N = 1$. We report average (a) MSE reconstruction loss during GA, and (b) downstream DFT accuracy over 3 seeds.

Rand-Top-K, trails `AE-PSL` by 5.3 pp. on average. Furthermore, to reach within 99% of no-compression DFT `AE-PSL` needs 12.4× less communication budget, with Rand-Top-K trailing by 13.9 pp. on average. This indicates that `AE-PSL` provides more communication-efficient convergence, as *its learned, feature-distribution-aware AE module preserves more task-relevant intermediate information with fewer transmitted bytes than heuristic compression modules.* Per-dataset results are reported in Appx. A.2.

5.3.2 Client-side Computational Overhead

We next evaluate whether the accuracy gains of `AE-PSL` come at excessive client-side computation cost. For this, we measure cumulative client-side GFLOPs during DFT, including the CSA warm-up cost of `AE-PSL`. As shown in Fig. 3 for CIFAR-100 with $N = 5$, `AE-PSL` consistently reaches a higher accuracy regime than the baselines across all compression levels. To reach $> 99\%$ of peak performance, the baselines require on average 1.07×, 1.15×, and 1.19× more GFLOPs than `AE-PSL` at low, mid, and high reduction levels, respectively. This shows that the lightweight encoder and CSA warm-up overhead are outweighed

by improved compute efficiency during DFT: as R increases, ***AE-PSL*** *reaches near-peak performance with fewer client-side GFLOPs by using learnable feature-preserving compression, whereas heuristic baselines require more computation and still converge to lower peak accuracies.* Results for the remaining datasets are reported in Appx. A.2, where the same pattern is observed.

5.3.3 Impact of AutoEncoder Architecture

We analyze the effect of AE architecture on reconstruction fidelity and downstream accuracy. For each design, we first perform General Alignment (GA; Sec.4.1.1) and measure the resulting MSE reconstruction loss, then integrate the aligned AE into DFT on Food101 to assess downstream accuracy. To isolate architectural effects from client heterogeneity and decoder aggregation, we use a single-client setting ($N = 1$).

As shown in Fig.4, convolutional AEs are less robust across compression levels, suggesting that reshaping transformer patch-token sequences into 2D grids often leads to lower downstream accuracy and higher reconstruction loss. For MLP-based AEs, depth primarily matters under stronger compression: the 3-layer MLP achieves the highest accuracy at extreme reduction levels (e.g., $R \approx 25$), with limited gains at lower compression. Conversely, 1-layer architectures remain close to the no-compression setting ($R = 1$) for $R < 10$, yet their accuracy drops sharply as R increases. The results also reveal a clear inverse relationship between GA reconstruction loss and downstream DFT accuracy, indicating that preserving split-layer features is critical for effective fine-tuning. Based on this trade-off, we select 2-layer-MLP-(d-d-d_z) as the default architecture, as it achieves strong reconstruction and downstream accuracy while increasing client-side parameters by only 1.6%. We further analyze the accuracy–parameter trade-off in Appx. A.7.2.

5.3.4 AE-PSL Components Analysis

We now ablate the core components of AE-PSL: General Alignment (GA), Client-Side Alignment (CSA), and AE freezing during DFT (FZ). As shown in Table 2, directly inserting a randomly initialized AE causes severe collapse under *global*

Table 2: Effect of AE-PSL's core components: General Alignment (GA), Client-Side Alignment (CSA), and AE freezing during DFT (FZ). Experiments conducted with ViT-B/32 [9] under *global* and *local* evaluation for $R = 15$ with $N \in \{5, 25\}$. We report average accuracy across 3 seeds; deltas denote relative performance to *no compression* baselines (R=1) for the same settings.

N	Method	*Global evaluation*			*Local evaluation*	R
		CIFAR100	**Food101**	**SUN397**	**FEMNIST**	
$N = 5$	No Compression	82.7 ± 0.4	69.1 ± 2.1	67.0 ± 1.2	87.6 ± 0.7	—
	AE	1.4 ± 0.3 (−81.3)	1.1 ± 0.2 (−68.0)	0.3 ± 0.2 (−66.7)	86.9 ± 2.0 (−0.7)	15×
	AE + GA	75.2 ± 0.4 (−7.5)	63.2 ± 2.8 (−5.9)	62.8 ± 0.4 (−4.2)	87.1 ± 1.1 (−0.5)	15×
	AE + GA + FZ	79.6 ± 0.9 (−3.1)	67.2 ± 3.2 (−1.9)	64.9 ± 0.6 (−2.1)	$\mathbf{88.0 \pm 0.6}$ (+0.4)	15×
	AE + GA + CSA + FZ (AE-PSL)	$\mathbf{82.2 \pm 0.7}$ (−0.5)	$\mathbf{67.6 \pm 2.5}$ (−1.5)	$\mathbf{65.6 \pm 0.7}$ (−1.4)	88.0 ± 0.7 (+0.4)	15×
$N = 25$	No Compression	82.0 ± 0.1	67.5 ± 0.1	64.5 ± 2.7	85.4 ± 1.3	—
	AE	1.1 ± 0.1 (−80.9)	1.3 ± 0.2 (−66.2)	0.5 ± 0.6 (−64.0)	84.3 ± 1.8 (−1.1)	15×
	AE + GA	74.2 ± 2.3 (−7.8)	61.8 ± 0.7 (−5.7)	61.2 ± 2.1 (−3.3)	$\mathbf{85.5 \pm 1.1}$ (+0.1)	15×
	AE + GA + FZ	79.0 ± 1.5 (−3.0)	65.7 ± 0.4 (−1.8)	63.4 ± 1.1 (−1.1)	84.8 ± 0.9 (−0.6)	15×
	AE + GA + CSA + FZ (AE-PSL)	$\mathbf{82.2 \pm 0.5}$ (+0.2)	$\mathbf{66.7 \pm 1.3}$ (−0.8)	$\mathbf{64.8 \pm 1.3}$ (+0.3)	84.1 ± 0.8 (−1.3)	15×

evaluation. Alternatively, we note that each component addresses a different source of instability. GA substantially restores global accuracy by aligning the AE with the pre-trained feature manifold before DFT, while AE freezing during DFT further improves performance by keeping the compression mapping stable during fine-tuning, preventing reconstruction quality drift. Finally, CSA adapts the encoder to client-specific feature distributions while retaining a single aggregated decoder, further closing the gap to no-compression settings. Together, these components make learnable compression compatible with off-the-shelf pre-trained models, approaching near-uncompressed performance while achieving substantial communication reduction. Appx. A.3 expands on *local* evaluation results. Appx. A.4 further analyzes components by visualizing AE reconstruction fidelity throughout DFT. Appx. A.5 details alternative AE alignment strategies.

6 Conclusion

We present `AE-PSL` a framework designed for communication-efficient distributed fine-tuning of large-scale FMs on resource-constrained edge devices. By integrating a lightweight, *learnable AE* into PSL, `AE-PSL` effectively compresses intermediate feature representations, substantially reducing the volume of data transmitted between clients and the server. Whereas existing learnable compression methods showed promising results, these are inherently incompatible with pre-trained models causing catastrophic forgetting due to feature misalignment. To mitigate this, we introduce 2-stage alignment. This process—comprising GA and CSA—tailors learnable compression to each clients' feature distribution prior to fine-tuning, without centralizing private data. We evaluate `AE-PSL` across 4 vision datasets, showing that feature-aligned learnable compression provides a more robust accuracy–communication trade-off than heuristic SL compression, achieving 10.2× communication reduction without accuracy degradation. Furthermore, `AE-PSL` accelerates distributed convergence, reaching 99% of peak accuracy using 12.4× less total communication volume than heuristic baseline methods. Crucially, this communication efficiency does not rely on excessive local computation; `AE-PSL` requires up to 1.14× fewer client-side GFLOPs to achieve near-peak performance compared to heuristic alternatives. By introducing communication overhead reduction while preserving downstream task accuracy, `AE-PSL` establishes a scalable and communication-efficient pathway for collaborative pre-trained model fine-tuning in resource-constrained edge environments.

Limitations & Future Work. Although conceptually our two-stage alignment protocol for learnable compression generalizes to other modalities, its performance across non-vision domains remains unevaluated. Additionally, `AE-PSL` currently relies on transmitting the `[CLS]` token uncompressed to guarantee downstream accuracy. While highly effective for image classification, this structural dependency limits the framework's broader architectural scope. Recognizing that not all tokens hold equal semantic importance, future work will explore token-adaptive compression via learnable importance scores. This approach aims to eliminate the dependency on uncompressed global tokens, paving the way for a more versatile, model-agnostic edge fine-tuning framework.

Acknowledgments

This work was supported by the AIMS5.0 project under grant agreement no. 101112089.

References

1. Alvetreti, F., Pomponi, J., Di Lorenzo, P., Scardapane, S.: Communication efficient split learning of vits with attention-based double compression. arXiv preprint arXiv:2509.15058 (2025)
2. Ayad, A., Renner, M., Schmeink, A.: Improving the communication and computation efficiency of split learning for iot applications. In: 2021 IEEE Global Communications Conference (GLOBECOM). pp. 01–06 (2021). https://doi.org/10.1109/GLOBECOM46510.2021.9685493
3. Bossard, L., Guillaumin, M., Van Gool, L.: Food-101–mining discriminative components with random forests. In: European conference on computer vision. pp. 446–461. Springer (2014)
4. Caldas, S., Wu, P., Li, T., Konečný, J., McMahan, H.B., Smith, V., Talwalkar, A.: LEAF: A benchmark for federated settings. CoRR (2018), http://arxiv.org/abs/1812.01097
5. Castiglia, T.J., Das, A., Wang, S., Patterson, S.: Compressed-vfl: Communication-efficient learning with vertically partitioned data. In: International conference on machine learning. pp. 2738–2766. PMLR (2022)
6. Chen, X., Wu, W., Ji, F., Lu, Y., Li, L.: Privacy-Aware Split Federated Learning for LLM Fine-Tuning over Internet of Things. IEEE Internet of Things Journal (99) (2025). https://doi.org/10.1109/JIOT.2025.3600269
7. Chen, X., Li, J., Fan, D., Chakrabarti, C.: HeteroSFL: Split Federated Learning With Heterogeneous Clients and Non-IID Data. IEEE Internet of Things Journal **12**(15), 30460–30474 (2025)
8. Cohen, R.A., Choi, H., Bajić, I.V.: Lightweight compression of neural network feature tensors for collaborative intelligence. In: 2020 IEEE International Conference on Multimedia and Expo (ICME). pp. 1–6 (2020). https://doi.org/10.1109/ICME46284.2020.9102797
9. Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., Houlsby, N.: An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In: International Conference on Learning Representations (ICLR) (2021), https://openreview.net/forum?id=YicbFdNTTy
10. Eshratifar, A.E., Esmaili, A., Pedram, M.: Bottlenet: A deep learning architecture for intelligent mobile cloud computing services. In: 2019 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED). pp. 1–6. IEEE (2019)
11. Fudala, T., Tsouvalas, V., Meratnia, N.: Fine-tuning multimodal transformers on edge: A parallel split learning approach. arXiv preprint arXiv:2502.06355 (2025)
12. Guo, S., Lu, Z., Lu, S., Cui, Y., Tang, X., Wu, J.: Split learning optimized for the medical field: Reducing communication overhead. In: 2024 IEEE International Conference on Bioinformatics and Biomedicine (BIBM). pp. 3226–3231 (2024). https://doi.org/10.1109/BIBM62325.2024.10822194
13. Gupta, O., Raskar, R.: Distributed learning of deep neural network over multiple agents. Journal of Network and Computer Applications **116**, 1–8 (2018)

14. Hsieh, C.Y., Chuang, Y.C., Wu, A.Y.: C3-sl: Circular convolution-based batch-wise compression for communication-efficient split learning. In: 2022 IEEE 32nd International Workshop on Machine Learning for Signal Processing (MLSP). pp. 1–6. IEEE (2022)
15. Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., et al.: Lora: Low-rank adaptation of large language models. ICLR **1**(2), 3 (2022)
16. Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. In: International Conference on Learning Representations (ICLR). San Diego, CA, USA (2015)
17. Krizhevsky, A., Hinton, G., et al.: Learning multiple layers of features from tiny images (2009)
18. Li, Z., Wu, S., Li, L., Zhang, S.: Energy-efficient split learning for fine-tuning large language models in edge networks. IEEE Networking Letters (2025)
19. Lin, Z., Hu, X., Zhang, Y., Chen, Z., Fang, Z., Chen, X., Li, A., Vepakomma, P., Gao, Y.: Splitlora: A split parameter-efficient fine-tuning framework for large language models. arXiv preprint arXiv:2407.00952 (2024)
20. Lin, Z., Zhang, Y., Chen, Z., Fang, Z., Chen, X., Vepakomma, P., Ni, W., Luo, J., Gao, Y.: HSplitLoRA: A Heterogeneous Split Parameter-Efficient Fine-Tuning Framework for Large Language Models . IEEE Transactions on Mobile Computing (01), 1–17 (Apr 5555). https://doi.org/10.1109/TMC.2026.3680521, https://doi.ieeecomputersociety.org/10.1109/TMC.2026.3680521
21. Lyu, X., Liu, S., Liu, J., Ren, C.: Scalable aggregated split learning for data-driven edge intelligence on internet-of-things. IEEE Internet of Things Magazine **6**(4), 124–129 (2023)
22. Ma, J., Lyu, X., Jiang, J., Cui, Q., Yao, H., Tao, X.: Splitfrozen: Split learning with device-side model frozen for fine-tuning llm on heterogeneous resource-constrained devices. IEEE Communications Magazine **64**(3) (2026). https://doi.org/10.48550/arxiv.2503.18986
23. Matsubara, Y., Callegaro, D., Singh, S., Levorato, M., Restuccia, F.: Bottlefit: Learning compressed representations in deep neural networks for effective and efficient split computing. In: 2022 IEEE 23rd International Symposium on a World of Wireless, Mobile and Multimedia Networks (WoWMoM). pp. 337–346 (2022). https://doi.org/10.1109/WoWMoM54355.2022.00032
24. McMahan, H.B., Moore, E., Ramage, D., Hampson, S., Arcas, B.A.y.: Communication-efficient learning of deep networks from decentralized data. Artificial Intelligence and Statistics (AISTATS) **54**, 1273–1282 (2017)
25. Mudvari, A., Vainio, A., Ofeidis, I., Tarkoma, S., Tassiulas, L.: Adaptive compression-aware split learning and inference for enhanced network efficiency. ACM Trans. Internet Technol. **24**(4) (Nov 2024). https://doi.org/10.1145/3687471, https://doi.org/10.1145/3687471
26. Oh, Y., Lee, J., Brinton, C., Jeon, Y.S.: Communication-efficient split learning via adaptive feature-wise compression. IEEE Transactions on Neural Networks and Learning Systems pp. 1–15 (01 2025). https://doi.org/10.1109/TNNLS.2025.3526227
27. Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M.S., Berg, A.C., Fei-Fei, L.: Imagenet large scale visual recognition challenge. CoRR (2014), http://arxiv.org/abs/1409.0575
28. Shao, J., Zhang, J.: Bottlenet++: An end-to-end approach for feature compression in device-edge co-inference systems. 2020 IEEE International Conference on Communications Workshops (ICC Workshops) pp. 1–6 (2019), https://api.semanticscholar.org/CorpusID:207757421

29. Shiranthika, C., Hadizadeh, H., Saeedi, P., Bajić, I.V.: SplitFedZip: Learned compression for data transfer reduction in split-federated learning (2024), https://arxiv.org/abs/2412.17150
30. Singh, A., Vepakomma, P., Gupta, O., Raskar, R.: Detailed comparison of communication efficiency of split learning and federated learning. arXiv preprint arXiv:1909.09145 (2019)
31. Xiao, J., Hays, J., Ehinger, K.A., Oliva, A., Torralba, A.: Sun database: Large-scale scene recognition from abbey to zoo. In: 2010 IEEE computer society conference on computer vision and pattern recognition. pp. 3485–3492. IEEE (2010)
32. Yurochkin, M., Agarwal, M., Ghosh, S., Greenewald, K., Hoang, N., Khazaeni, Y.: Bayesian nonparametric federated learning of neural networks. In: International conference on machine learning. pp. 7252–7261. PMLR (2019)
33. Zhang, S., Cheng, G., Li, Z., Wu, W.: Splitllm: Hierarchical split learning for large language model over wireless network. In: 2024 IEEE Globecom Workshops (GC Wkshps). pp. 1–6. IEEE (2024)
34. Zhang, S., Cheng, G., Wu, W., Xinyu, H., Song, L., Shen, X.: Split fine-tuning for large language models in wireless networks. IEEE Journal of Selected Topics in Signal Processing pp. 1–16 (01 2025). https://doi.org/10.1109/JSTSP.2025.3581484
35. Zhao, K., Zhu, C., Chen, M., Chongwen, H., Yang, Z., Zhang, Z.: Sflllm: Efficient split federated learning for large language model over wireless networks. pp. 1835–1840 (12 2025). https://doi.org/10.1109/GLOBECOM59602.2025.11432069
36. Zhao, W.X., Zhou, K., Li, J., Tang, T., Dong, Z., Hou, Y., Zhang, B., Min, Y., Zhang, J., Liu, P., Wang, X., Du, Y., Yang, C., Chen, Y., Chen, Z., Jiang, J., Ren, R., Li, Y., Tang, X., Liu, Z., Hu, Y., Nie, J.Y., Wen, J.R.: A survey of large language models. Frontiers of Computer Science **20**(12), 2012627 (2026). https://doi.org/10.1007/s11704-026-60308-3
37. Zhao, Y., Li, M., Lai, L., Suda, N., Civin, D., Chandra, V.: Federated learning with non-iid data. CoRR (2018), http://arxiv.org/abs/1806.00582
38. Zheng, F., Chen, C., Lyu, L., Yao, B.: Reducing communication for split learning by randomized top-k sparsification. In: Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence. IJCAI '23 (2023). https://doi.org/10.24963/ijcai.2023/519, https://doi.org/10.24963/ijcai.2023/519

A Appendix

A.1 Detailed Experimental Setup

We evaluate `AE-PSL` on public vision classification datasets. CIFAR-100 [17], Food101 [3] and SUN397 [31] do not provide per-client test sets, and are used for *global* evaluation. If a dataset is not pre-split, 20% is left out for evaluation. We partition client data using Dirichlet sampling [32] with $\alpha = 0.5$. Under *local evaluation*, we use FEMNIST [4], a dataset specifically collected for distributed learning settings, providing per-user datasets. For *local evaluation* on FEMNIST, the dataset is inherently partitioned by `writer_id`, with the total pool of writers distributed evenly and in a non-overlapping manner across the participating clients. To manage computational constraints, we subsample 10% of the data samples from each assigned writer before applying localized train and test splits.

Training Parameters. DFT is performed for 10 epochs using LoRA [15] with rank $r = 16$ and $\alpha = 32$, optimized via Cross Entropy loss using Adam [16] with learning rate 10^{-4}, which is scheduled using Cosine Annealing, bringing the rate down to 5×10^{-5} over the duration of DFT. The global batch size is fixed at 125, hence client batch size are $25, 5$ for $N = 5, 25$ respectively. For GA, the AE is trained on ImageNet100 [27] for 30 epochs using Adam with with learning rate 10^{-4} with a Cosine Annealing scheduler annealing the rate down to 5×10^{-5} over 30 epochs. Batch size is fixed at 512. In the case of CSA, the client-side model partition remains entirely frozen, except for the AE. The AE module is optimized using Adam, with a higher learning rate of 10^{-3} over a single epoch. Client batch size are equal to the DFT client batch size.

Baselines. In order to ensure fair comparison, we fix 3 communication reduction levels (R) for our method and all baselines. This accounts for optional compression of the `[CLS]` token, which is disabled if it improves performance for a given baseline. Specific parameters setting for each method are given in Table 3. Each method performs DFT 3 times. All reported values are averaged over these 3 runs.

Table 3: Compression-parameter settings used for each method across communication-reduction regimes. CLS tokens are left uncompressed unless indicated otherwise.

Method	**Parameter**	**Communication Reduction (R)**			**CLS Compr.**
		Low ($\sim 5\times$)	**Mid** ($\sim 10\times$)	**High** ($\sim 20\times$)	
AE-PSL (Ours)	Latent dimension	140	64	24	No
Rand-Top-K [38]	Sparsity	0.15	0.065	0.025	No
C3-SL [14]	Batch-wise CR	5	12	34	No
ADC [1]*	Batch/token CR	$1/\sqrt{5}$	$1/\sqrt{10}$	$1/\sqrt{20}$	Yes

A.2 Supplementary Performance Comparison with Baselines

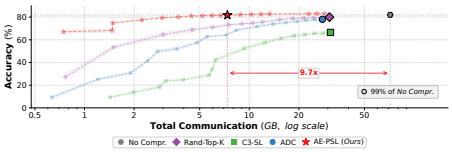
To evaluate performance of `AE-PSL` compared to the established baselines, we measure top-1 accuracy at varying degrees of communication reduction. Table 4 contains results with standard deviations included. Our proposal achieves the highest performance in terms of accuracy across all baselines.

Table 4: Performance under ***global*** and ***local*** evaluation using ViT-B/32 [9] across datasets for $N \in \{5, 25\}$ and $R \in \{5, 10, 20\}$. We report average accuracy across 3 seeds including standard deviations, whereas *deltas* denote relative performance to *no compression* baseline (R=1) for the same settings.

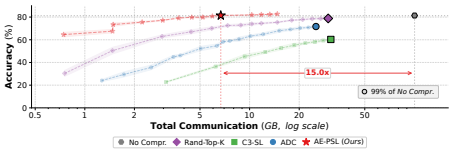
	Dataset	Method	$N = 5$			$N = 25$		
			Low ($R \approx 5$)	Mid ($R \approx 10$)	High ($R \approx 20$)	Low ($R \approx 5$)	Mid ($R \approx 10$)	High ($R \approx 20$)
Global Eval.	**CIFAR100**	No Compr. ($R = 1$)		82.7 ± 0.2			82.0 ± 0.1	
		C3-SL	66.5 ± 1.3 (−16.2)	50.9 ± 1.2 (−31.8)	37.3 ± 0.2 (−45.4)	60.0 ± 0.5 (−22.0)	60.3 ± 1.0 (−21.7)	59.9 ± 1.0 (−22.1)
		Rand-Top-K	79.9 ± 0.5 (−2.8)	76.2 ± 0.4 (−6.5)	61.5 ± 2.9 (−21.2)	78.8 ± 1.2 (−3.2)	75.4 ± 0.6 (−6.6)	63.3 ± 1.2 (−18.7)
		ADC	78.0 ± 0.3 (−4.7)	68.4 ± 0.4 (−14.3)	53.7 ± 1.0 (−29.0)	71.7 ± 1.6 (−10.3)	68.0 ± 1.8 (−14.0)	63.4 ± 1.6 (−18.6)
		`AE-PSL` (Ours)	$\mathbf{83.0 \pm 0.0}$ (+0.3)	$\mathbf{82.7 \pm 0.2}$ (+0.0)	$\mathbf{81.4 \pm 0.3}$ (−1.3)	$\mathbf{82.7 \pm 0.3}$ (+0.7)	$\mathbf{82.7 \pm 0.4}$ (+0.7)	$\mathbf{81.1 \pm 0.2}$ (−0.9)
	Food101	No Compr. ($R = 1$)		69.1 ± 0.9			67.5 ± 0.1	
		C3-SL	37.0 ± 2.2 (−32.1)	40.3 ± 1.2 (−28.8)	32.1 ± 1.2 (−37.0)	31.3 ± 3.0 (−36.2)	31.3 ± 3.0 (−36.2)	31.3 ± 3.0 (−36.2)
		Rand-Top-K	66.5 ± 1.5 (−2.6)	60.2 ± 0.2 (−8.9)	35.7 ± 4.0 (−33.4)	65.1 ± 1.2 (−2.4)	60.3 ± 0.7 (−7.2)	43.1 ± 2.6 (−24.4)
		ADC	63.3 ± 1.4 (−5.8)	54.6 ± 1.6 (−14.5)	44.0 ± 1.3 (−25.1)	56.2 ± 1.3 (−11.3)	51.8 ± 1.5 (−15.7)	47.1 ± 1.1 (−20.4)
		`AE-PSL` (Ours)	$\mathbf{70.3 \pm 1.2}$ (+1.2)	$\mathbf{69.6 \pm 1.2}$ (+0.5)	$\mathbf{65.3 \pm 1.7}$ (−3.8)	$\mathbf{68.7 \pm 0.5}$ (+1.2)	$\mathbf{68.1 \pm 0.9}$ (+0.6)	$\mathbf{64.3 \pm 1.3}$ (−3.2)
	SUN397	No Compr. ($R = 1$)		67.0 ± 0.6			64.5 ± 1.2	
		C3-SL	48.0 ± 1.1 (−19.0)	41.3 ± 2.1 (−25.7)	31.6 ± 2.1 (−35.4)	41.5 ± 1.2 (−23.0)	41.5 ± 1.2 (−23.0)	41.5 ± 1.2 (−23.0)
		Rand-Top-K	66.3 ± 0.7 (−0.7)	62.9 ± 1.0 (−4.1)	48.5 ± 3.1 (−18.5)	64.6 ± 0.7 (+0.1)	61.2 ± 1.4 (−3.3)	50.0 ± 0.4 (−14.5)
		ADC	62.2 ± 0.5 (−4.8)	52.9 ± 1.1 (−14.1)	39.0 ± 1.7 (−28.0)	55.5 ± 0.6 (−9.0)	52.1 ± 0.8 (−12.4)	47.5 ± 1.4 (−17.0)
		`AE-PSL` (Ours)	$\mathbf{67.4 \pm 0.4}$ (+0.4)	$\mathbf{66.7 \pm 0.3}$ (−0.3)	$\mathbf{64.8 \pm 0.2}$ (−2.2)	$\mathbf{65.2 \pm 1.0}$ (+0.7)	$\mathbf{65.0 \pm 0.5}$ (+0.5)	$\mathbf{63.8 \pm 0.5}$ (−0.7)
Local Eval.	**FEMNIST**	No Compr. ($R = 1$)		87.9 ± 0.3			85.8 ± 0.6	
		C3-SL	85.2 ± 0.8 (−2.7)	83.5 ± 0.1 (−4.4)	81.4 ± 0.5 (−6.5)	80.7 ± 1.1 (−5.1)	39.8 ± 4.0 (−46.0)	6.9 ± 1.8 (−78.9)
		Rand-Top-K	87.6 ± 0.5 (−0.3)	87.2 ± 0.4 (−0.7)	86.4 ± 0.2 (−1.5)	84.6 ± 0.7 (−1.2)	83.8 ± 0.4 (−2.0)	82.2 ± 0.3 (−3.6)
		ADC	86.6 ± 1.2 (−1.3)	81.1 ± 0.9 (−6.8)	71.8 ± 4.3 (−16.1)	82.4 ± 1.4 (−3.4)	78.1 ± 3.4 (−7.7)	73.8 ± 5.4 (−12.0)
		`AE-PSL` (Ours)	$\mathbf{88.1 \pm 0.3}$ (+0.2)	$\mathbf{88.0 \pm 0.1}$ (+0.1)	$\mathbf{87.9 \pm 0.5}$ (+0.0)	$\mathbf{85.9 \pm 0.4}$ (+0.1)	$\mathbf{85.5 \pm 0.9}$ (−0.3)	$\mathbf{84.6 \pm 1.0}$ (−1.2)

Performance under Fixed Communication Budget. Here, we further evaluate the total communication overhead across clients on all datasets, shown in Fig. 5. The observed patterns remain consistent across all datasets.

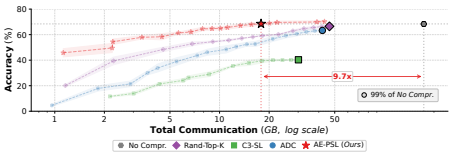
Client-side Computational Overhead. Here we evaluate the total communication overhead across clients. In Fig. 6 plots for all datasets and client counts are presented.



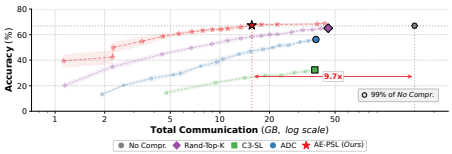
(a) **CIFAR-100** / $N = 5$



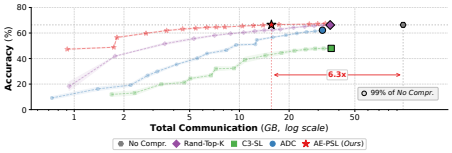
(b) **CIFAR-100** / $N = 25$



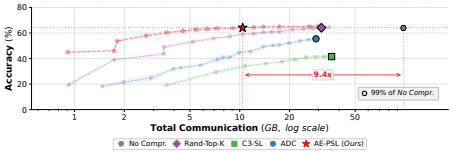
(c) **Food101** / $N = 5$



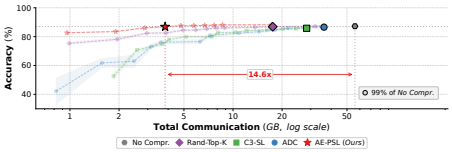
(d) **Food101** / $N = 25$



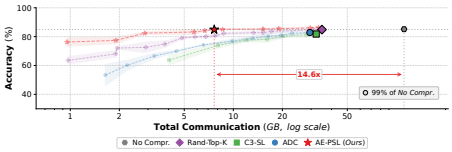
(e) **SUN397** / $N = 5$



(f) **SUN397** / $N = 25$

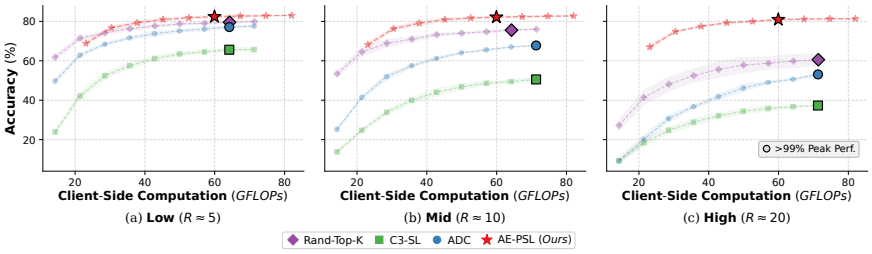


(g) **FEMNIST** / $N = 5$

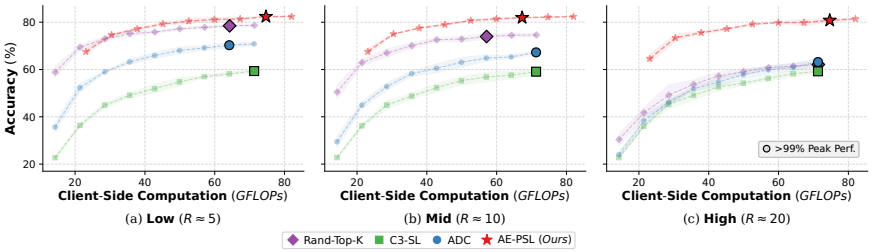


(h) **FEMNIST** / $N = 25$

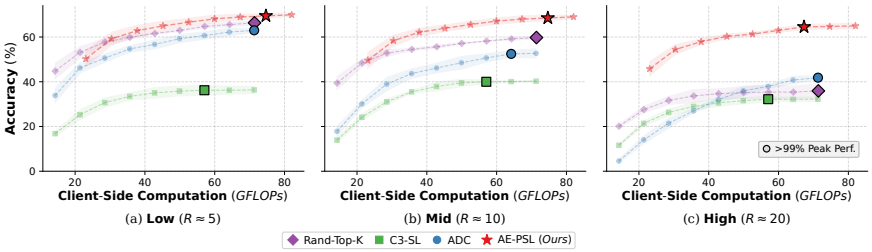
Fig. 5: Accuracy–communication trade-off using ViT-B/32 for $N = 5$ and $N = 25$ across datasets. We report average accuracy across 3 seeds for different communication volumes (GB); markers indicate when a method reaches >95% of peak performance.



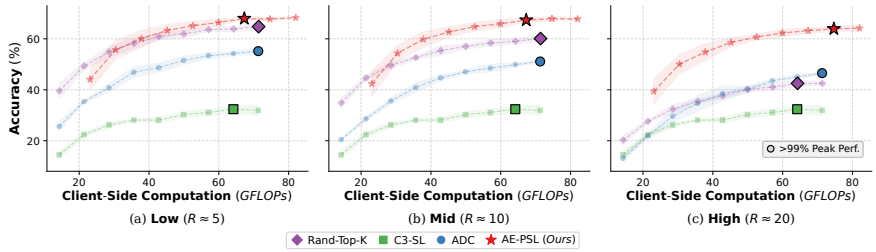
(a) **CIFAR-100** / $N = 5$



(b) **CIFAR-100** / $N = 25$

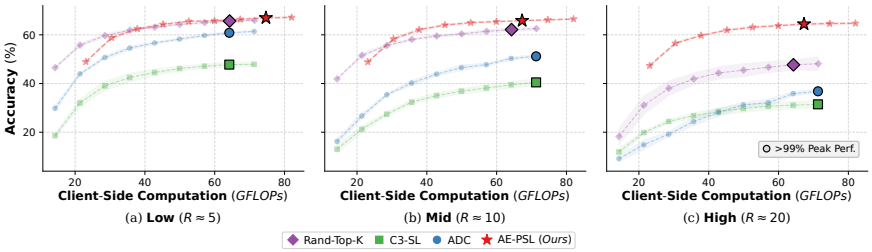


(c) **Food101** / $N = 5$

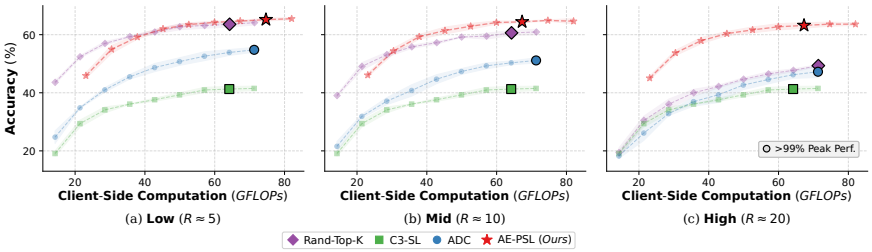


(d) **Food101** / $N = 25$

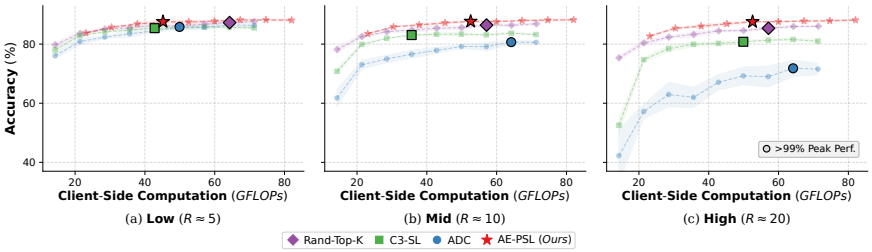
Fig. 6: Accuracy–client-side compute trade-off using ViT-B/32 for $N \in \{5, 25\}$ and $R \in \{5, 10, 20\}$ across datasets. We report average accuracy across 3 seeds for cumulative client-side computational overhead (GFLOPs) across R levels; markers indicate when a method reaches >95% of peak performance. (Continued on next page)



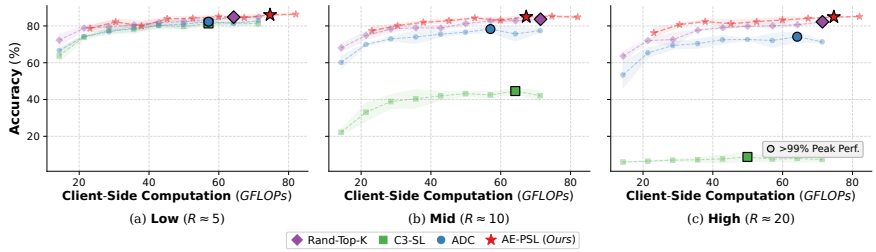
(e) **SUN397** / $N = 5$



(f) **SUN397** / $N = 25$



(g) **FEMNIST** / $N = 5$



(h) **FEMNIST** / $N = 25$

Fig. 6: Accuracy–client-side compute trade-off using ViT-B/32 for $N \in \{5, 25\}$ and $R \in \{5, 10, 20\}$ across datasets. We report average accuracy across 3 seeds for cumulative client-side computational overhead (GFLOPs) across R levels; markers indicate when a method reaches >95% of peak performance.

A.3 Additional Analysis of Local Evaluation

As shown in Table 5, downstream performance is largely preserved when a randomly initialized AE is introduced, provided the module remains active during inference. Conversely, bypassing the AE at test time precipitates a severe degradation in accuracy. This occurs because the primary model and the AE co-adapt during DFT; removing the compression module subsequently induces a substantial feature distribution misalignment.

Table 5: Effect of the AE module during FEMNIST *local evaluation*. We report average accuracy across 3 seeds at $R \approx 15$, with the AE active at inference time (*w/ AE*) and removed at inference time (*w/o AE*); deltas denote relative performance with respect to the *no compression* setting ($R = 1$).

Method	**N = 5**		**N = 25**		**R**
	Local Eval. (w/ AE)	*Local Eval. (w/o AE)*	*Local Eval. (w/ AE)*	*Local Eval. (w/o AE)*	
No Compression	87.6 ± 0.7		85.4 ± 1.3		—
AE	86.9 ± 2.0 (−0.7)	1.4 ± 1.3 (−86.2)	84.3 ± 1.8 (−1.1)	1.8 ± 3.3 (−83.6)	15×
AE + GA	87.1 ± 1.1 (−0.5)	54.3 ± 12.3 (−33.3)	**85.5 ± 1.1** (**+0.1**)	54.5 ± 11.8 (−30.9)	15×
AE + GA + FZ	**88.0 ± 0.6** (**+0.4**)	81.9 ± 0.7 (−5.7)	84.8 ± 0.9 (−0.6)	**81.3 ± 2.8** (**−4.1**)	15×
AE + GA + CSA + FZ (AE-PSL)	88.0 ± 0.7 (+0.4)	**82.9 ± 2.2** (**−4.7**)	84.1 ± 0.8 (−1.3)	80.5 ± 1.6 (−4.9)	15×

A.4 Additional Analysis of `AE-PSL` Components

To gain a deeper understanding of why `AE-PSL` components improve downstream task accuracy, we measure AE reconstruction fidelity throughout DFT. We define this fine-tuning reconstruction error as $\mathcal{E}_{\text{rec}} = \|\mathbf{H}_n - \widehat{\mathbf{H}}_n\|_2^2$, which computes the mean squared error between the original client-side patch activations $\mathbf{H}_n$ and the server-side reconstructions $\widehat{\mathbf{H}}_n$, averaged across clients for a given batch. We distinguish $\mathcal{E}_{\text{rec}}$ from the optimization objective used during the GA phase, as $\mathcal{E}_{\text{rec}}$ serves strictly as an evaluation metric during DFT.

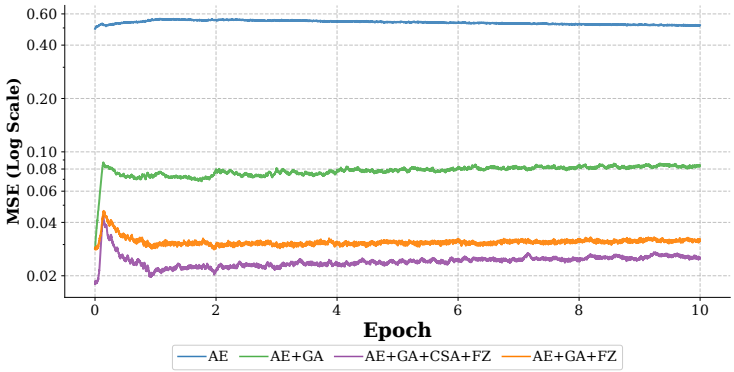


Fig. 7: Per-client averaged reconstruction error ($\mathcal{E}_{\text{rec}}$) during DFT for $N = 5$ on CIFAR-100.

As evidenced in Fig. 7, inserting a randomly initialized AE causes high initial $\mathcal{E}_{\text{rec}}$. The addition of the GA phase (Sec. 4.1.1) significantly reduces the initial error $\mathcal{E}_{\text{rec}}$, however rapidly increases as DFT progresses if the AE parameters are left unfrozen. By freezing (FZ) the AE during DFT, $\mathcal{E}_{\text{rec}}$ is relatively stable. This indicates that updating the AE weights concurrently with the task optimization (via Cross Entropy loss) yields gradients that counter-intuitively degrade reconstruction fidelity. We hypothesize that while the primary model weights and the AE may co-adapt—causing the latent space to drift—this adaptation is detrimental under the *global* evaluation, where the AE is explicitly bypassed during inference. Finally, the application of CSA (Sec. 4.1.2) establishes an even lower $\mathcal{E}_{\text{rec}}$ at the onset of DFT, directly correlating with the increased downstream task accuracy observed in our main results.

A.5 Alternative AE Alignment Strategies.

Beyond the two-stage alignment procedure detailed in Sec. 4.1, we investigated several alternative training strategies aimed directly at minimizing $\mathcal{E}_{\text{rec}}$ during fine-tuning.

Encoder-Only Client-Specific Alignment (EO-CSA). To eliminate the communication overhead associated with server-side decoder aggregation of standard CSA, we evaluated an Encoder-Only Client-Specific Alignment strategy. In this configuration, only the client-side encoder $E_{\boldsymbol{\phi}_n}$ is updated during the local warm-up phase, while the server-side decoder $D_{\boldsymbol{\psi}}$ remains strictly frozen. As demonstrated in Fig. 8 and Table 6, EO-CSA yields negligible AE adaptation; the encoder struggles to map local data distributions to the existing latent space effectively without corresponding weight updates in the decoder.

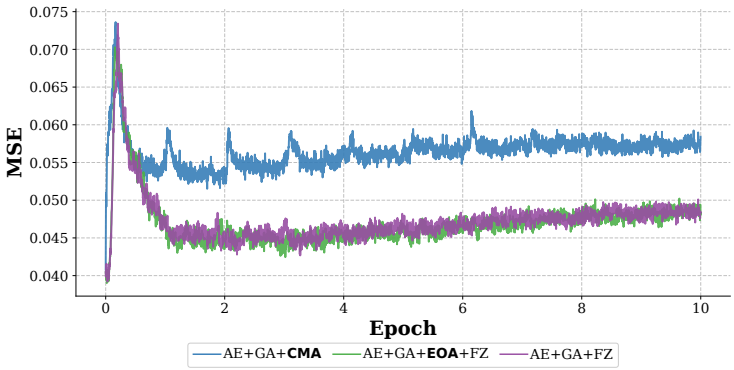


Fig. 8: Per-client averaged reconstruction error ($\mathcal{E}_{\text{rec}}$) during DFT for $N = 5$ on CIFAR-100.

Concurrent MSE Alignment (CMA). To address the drift in reconstruction error observed during DFT without freezing, we evaluate Concurrent MSE Alignment (CMA). Instead of freezing the compression module entirely, we maintained a frozen local copy of the global server-side decoder $D_{\boldsymbol{\psi}^{\text{global}}}$ on the

Table 6: Effect of alternative AE alignment strategies: Encoder-Only Client-Specific Alignment (EO-CSA), Concurrent MSE Alignment (CMA), and iterative Client-Specific Alignment (ℓ-CSA). Experiments are conducted with ViT-B/32 under *global* evaluation for $R = 15$ with $N = 5$ on CIFAR100. We report average accuracy across 3 seeds.

AE Alignment Strategy	**Accuracy (%)**
AE + GA + FZ	79.6 ± 0.9
AE + GA + EO-CSA + FZ	79.6 ± 0.7
AE + GA + CMA	78.1 ± 1.1
AE + GA + 1-CSA + FZ (`AE-PSL`)	82.2 ± 0.7
AE + GA + 2-CSA + FZ	82.4 ± 0.9
AE + GA + 3-CSA + FZ	$\mathbf{82.4 \pm 0.8}$

client. During the forward pass, an auxiliary MSE loss was computed locally to update the encoder E_{ϕ_n} concurrently with the primary task optimization. The regular cross entropy loss assumes the AE as an identity function in CMA. However, CMA underperformed compared to the strictly frozen AE approach, both in terms of $\mathcal{E}_{\text{rec}}$ and downstream task accuracy as visualized in Fig. 8 and Table 6. This performance degradation likely stems from conflicting optimization signals: the main cross entropy gradient implicitly assumes a static mapping across the split layer, while the auxiliary MSE gradient continuously perturbs the encoder's latent space. This finding highlights that adapting the decoder to the latent projections is critical for preserving reconstruction fidelity, validating our choice to utilize a client-side warm-up combined with server-side decoder aggregation.

Iterative Client-Specific Alignment (ℓ-CSA). We explored extending the standard single-round CSA (equivalent to 1-CSA) into an Iterative Client-Specific Alignment (ℓ-CSA) process spanning multiple epochs. Under ℓ-CSA, local client-specific alignment and subsequent server-side decoder aggregation are repeated for ℓ epochs, where the aggregated global decoder is redistributed to all clients at the end of each epoch. As illustrated in Fig. 9 and Table 6, this iterative approach yielded a marginal reduction in reconstruction error and an insignificant increase in downstream task accuracy compared to standard 1-CSA. Consequently, the substantial increase in client-side computational load required for ℓ client-side epochs renders ℓ-CSA inefficient relative to the minimal performance gains achieved.

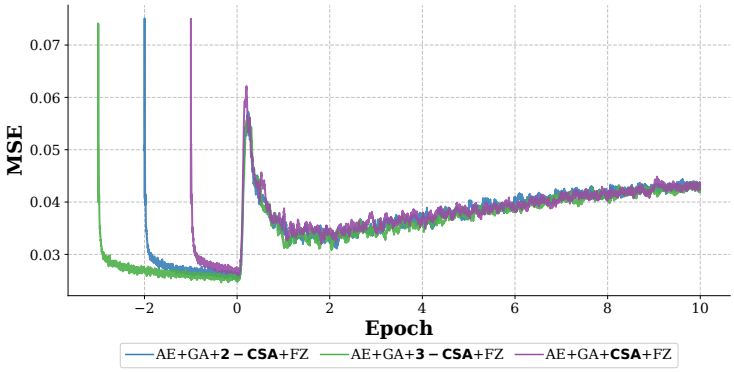


Fig. 9: Per-client averaged reconstruction error ($\mathcal{E}_{\text{rec}}$) of the AE during DFT, for $N = 5$ on CIFAR-100. The epochs of the ℓ-CSA phase are visualized as negative epochs. Note that no server-side training occurs during the CSA phase.

A.6 Impact of Split Layer Selection

In order to analyze the optimal split layer s we training multiple dedicated AEs via GA (as detailed in Sec. 4.1.1). For each candidate layer $0 \leq s \leq 6$, an AE is aligned on intermediate activations extracted from the public ImageNet100 dataset. Subsequently each AE is integrated into the model at its respective split point to evaluate downstream top-1 accuracy via DFT.

As illustrated in Fig. 10, the choice of split layer significantly influences on downstream performance, showing distinct trends between datasets. Notably, for Food101, positioning the bottleneck at early layers ($s \in \{1, 2\}$) induces substantial performance degradation. On the other hand, while the average downstream accuracy for CIFAR-100 remains stable at these early layers, we observe a significant increase in variance, indicating that early split layers introduce training instability.

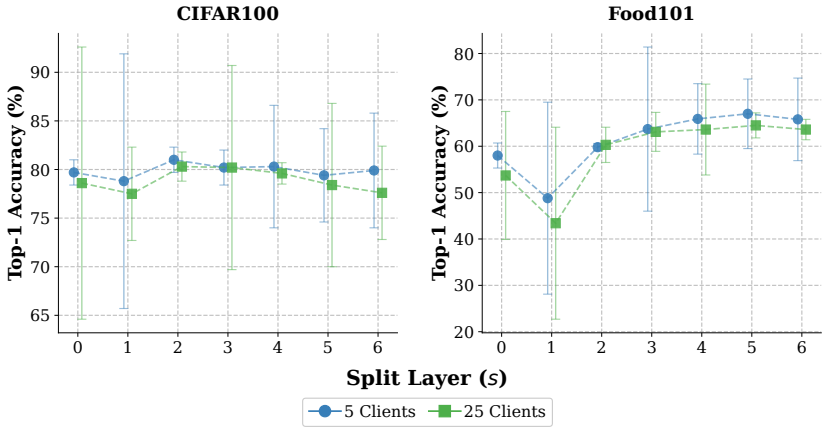


Fig. 10: Accuracy of DFT with AE compression at split layer s, evaluated on ViT-B\32 which consists of 12 layers in total. Communication reduction level is fixed ($R \approx 15$). Data points are slightly offset to enhance readability of the error intervals.

Configuring a lower split layer is advantageous because it offloads a larger portion of the model to the server, thereby minimizing client-side memory and compute requirements. However, our evaluation suggest that no clear universally optimal split point exists across evaluated settings. Consequently, to balance client-side computational overhead against training stability and task accuracy, $s = 5$ was selected as the default configuration for all experiments.

A.7 AutoEncoder Architecture & Computational Overhead Analysis

This section details supplementary analysis of architectural configurations for the AE. The experimental protocol evaluates each AE architecture by first executing a GA phase (Sec. 4.1.1) to measure the GA reconstruction loss (MSE) for a given communication reduction level R (which determines d_z). Following this alignment, each AE is integrated into DFT with $N = 1$ client to assess its impact on downstream top-1 accuracy.

A.7.1 Activation Functions in AE architectures To justify the selection of the activation function, we compare two architecturally identical models differing only in their non-linearity: ReLU and GELU. As illustrated in Fig. 11, the GELU-based architecture consistently yields a lower GA reconstruction loss. Regarding downstream top-1 accuracy, GELU maintains a marginal advantage, albeit with slightly higher variance. Given its superior reconstruction fidelity and minor improvement in task performance, GELU is adopted across all AE architectures in our primary experiments.

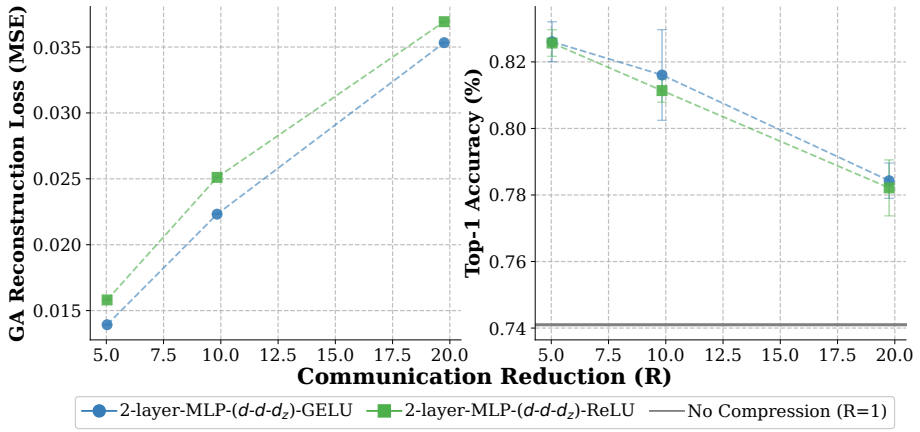


Fig. 11: Evaluation of AE architectures varying communication reduction levels (R) on Food101 ($N = 1$). Left: GA reconstruction loss (MSE) Right: Corresponding DFT top-1 accuracy.

A.7.2 AE Parameter vs. Accuracy Trade-off. For the majority of evaluated AE architectures, the additional computational overhead remains marginal relative to the client-side model ($< 2\%$ parameter increase), assuming a split layer of $s = 5$ for ViT-B/32. Nevertheless, to accommodate edge scenarios with stricter computational constraints, we analyze the compression performance of various AE architectures as a function of their parameter count. The parameter count for a given architecture scales dynamically, as the latent dimension d_z dictates the size of the final hidden layer.

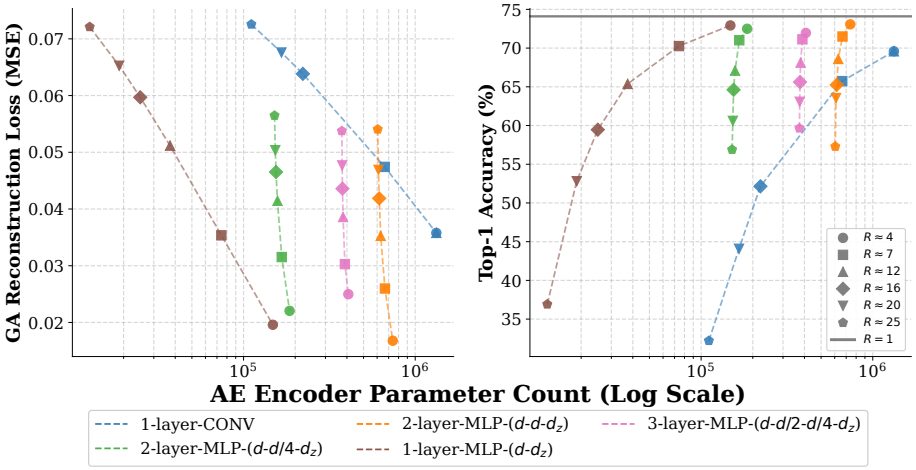


Fig. 12: Evaluation of encoder parameter counts for AE architectures across varying communication reduction levels (R) on the Food101 dataset ($N = 1$). Only the encoder parameter count is denoted, as client-side computation is our primary focus; the decoder is symmetric and of equal size. Left: GA reconstruction loss (MSE). Right: Corresponding DFT top-1 accuracy.

As depicted in Fig. 12, the 2-layer-MLP-(d-d-d_z) configuration sustains high accuracy across most compression ratios. The deeper 3-layer-MLP-(d-d/2-d/4-d_z) variant demonstrates improved performance specifically under extreme compression ($R \approx 25$). Notably, increasing the parameter count does not strictly correlate with improved downstream task accuracy; however, the expanded model capacity does reduce reconstruction MSE at higher compression levels. Conversely, highly parameterized architectures yield diminishing returns at lower compression ratios. As such, the 2-layer-MLP-(d-d-d_z) structure offers an optimal trade-off for maximizing fine-tuning accuracy, introducing only a 1.6% increase in client-side parameters. For environments governed by strict computational limitations, the 2-layer-MLP-(d-d/4-d_z) architecture provides viable performance while reducing the parameter count by an order of magnitude.

A.8 Impact of [CLS] Token Compression.

The [CLS] token aggregates global contextual information that is critical for downstream classification at the server-side layers. Consequently, maintaining its representation fidelity is vital for model convergence. To quantify the impact of compressing this feature, we evaluate the top-1 accuracy under fixed communication reduction levels (R). To ensure a rigorous comparison, when the [CLS] token is compressed, the latent dimension d_z of the bottleneck is increased proportionally to match the exact communication reduction level of the uncompressed-[CLS] baseline.

As demonstrated in Table 7, subjecting the [CLS] token to compression induces a severe drop in downstream task performance. This degradation is highly pronounced across all communication reduction levels and further degrades as the reduction ratio increases. These findings validate our framework design choice to transmit the [CLS] token uncompressed, prioritizing its semantic integrity to stabilize fine-tuning over edge networks. While preserving the [CLS] token intact guarantees training stability, it leaves the framework reliant on this specific architectural design choice.

Table 7: Impact of [CLS] token compression on DFT top-1 accuracy across degrees of communication reduction (R).

Compress [CLS]	**Low** ($R \approx 5$)	**Mid** ($R \approx 10$)	**High** ($R \approx 20$)
False (AE-PSL)	**82.4 ± 0.5**	**82.0 ± 0.5**	**79.0 ± 0.8**
True	73.9 ± 4.5	62.6 ± 2.5	46.8 ± 31.1